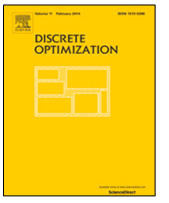
Discrete Optimization 44 (2022) 100594



Contents lists available at ScienceDirect

Discrete Optimization

www.elsevier.com/locate/disopt



Penalty and partitioning techniques to improve performance of QUBO solvers



Amit Verma *, Mark Lewis

Craig School of Business, Missouri Western State University, Saint Joseph, MO, 64507, United States

ARTICLE INFO

Article history:
Received 20 December 2019
Received in revised form 19 May 2020
Accepted 18 June 2020
Available online 29 June 2020

Keywords:
Quadratic Unconstrained Binary Optimization
Nonlinear optimization
Pseudo-Boolean optimization
Equality constraint
Inequality constraint

ABSTRACT

Quadratic Unconstrained Binary Optimization (QUBO) modeling has become a unifying framework for solving a wide variety of both unconstrained as well as constrained optimization problems. More recently, QUBO (or equivalent $-1/+1$ Ising Spin) models are a requirement for quantum annealing computers. Noisy Intermediate-Scale Quantum (NISQ) computing refers to classical computing preparing or compiling problem instances for compatibility with quantum hardware architectures. The process of converting a constrained problem to a QUBO compatible quantum annealing problem is an important part of the quantum compiler architecture and specifically when converting constrained models to unconstrained the choice of penalty magnitude is not trivial because using a large penalty to enforce constraints can overwhelm the solution landscape, while having too small a penalty allows infeasible optimal solutions. In this paper we present NISQ approaches to bound the magnitude of the penalty scalar M and demonstrate efficacy on a benchmark set of problems having a single equality constraint and present a QUBO partitioning approach validated by experimentation.

Published by Elsevier B.V.

1. Introduction

Quadratic Unconstrained Binary Optimization (QUBO) problems have emerged as a popular unifying framework for modeling a wide variety of combinatorial optimization problems (see [1] for details). A QUBO model is of form $min\ x'Qx$ where x represents the binary decision vector and by definition, a QUBO model cannot contain any constraints. However, any equality constraint of the form $Ax = b$ can be transformed into an objective function penalty term $M(Ax - b)^2$ where in a minimization problem M represents a large positive number. Current practice is to estimate M by making it larger than an arbitrary element of Q, which generally works, but may fail. Sometimes, M is chosen through domain expertise [2]. NISQ-era computers currently have less numeric precision and dynamic range and are thus significantly impacted by the magnitude of the penalty coefficient M [3] which we address with a novel approach to a lower bound estimate for M. We explore the impact of this lower bound using benchmark QUBO problems.

* Corresponding author.
E-mail address: averma@missouriwestern.edu (A. Verma).

https://doi.org/10.1016/j.disopt.2020.100594
1572-5286/Published by Elsevier B.V.

Moreover, any inequality constraint $Ax \leq b$ can be transformed to an equality constraint $Ax + Ds = b$ where s represents the slack variables and D represents the coefficients of the slack variables. Minimizing the number of slack variables is desirable in order to decrease the size and complexity of the transformed problem because increasing the number of slack variables increases the number of quadratic QUBO terms which are the components that tend to create computational difficulties. In this paper, we propose a partitioning approach in Section 3 to divide the slack range of the inequality constraint thereby reducing the usage of slack variables. We illustrate the efficacy of this approach on benchmark QUBO datasets in Section 4.2.

To align with NISQ-era objectives of integrating the benefits of classical computing with quantum computing our goal is to find effective ways to handle a single linear equality constraint and a single linear inequality constraint in a QUBO model [4,5]. Related work includes [6–9] that involve mapping of constraints into quantum annealing hardware. For the unconstrained case, the impact of penalty magnitude and the number of auxiliary variables on the computational performance of fourth degree pseudo-Boolean functions has been explored in [10]. Our paper is organized as follows. We present our lower bound estimate of M in Section 2. The details of our partitioning approach to handle inequality constraint are presented in Section 3. Section 4 describes our 1-flip solver and examines the results on problems from a benchmark QUBO dataset. Conclusions are provided in Section 5.

2. Estimate of lower bound of M

The base problem of interest is $min\ x'Qx\ s.t.\ Ax = b,\ x \in \{0,1\}$. The standard transformation of this model to QUBO converts the constraints to the objective via penalties to $min\ x'Qx + M(Ax-b)^2$ and we investigate reducing the magnitude of M. Our estimate for lower bound of M is valid for any f-flip solver (although most existing solvers use a 1-bit flip to move from one solution to the next and our experiments will be based on 1-flips). Any f-flip solver for QUBO transitions from a solution vector x_{from} to a solution vector x_{to} (where x_{from} and x_{to} differ by exactly f bits) and during this transition, the following lemma hold:

Lemma 1. *For a minimization problem with $x'_{from}Qx_{from} > x'_{to}Qx_{to}$, $Ax_{from} = b$, and $Ax_{to} \neq b$, the penalty coefficient magnitude M should satisfy the following inequality:*

$$x'_{from}Qx_{from} < x'_{to}Qx_{to} + M(Ax_{to} - b)^2 \tag{1}$$

Proof. The proof is obvious through contradiction. If the converse was true i.e.

$$x'_{from}Qx_{from} > x'_{to}Qx_{to} + M(Ax_{to} - b)^2$$

then the f-flip solver would prefer the solution x_{to} compared to x_{from} in a minimization problem. We would not want such behavior from a greedy local search solver since x_{to} is infeasible even though the objective function is lower. M should be chosen such that these moves are restricted. This would result in a contradiction. □

Eq. (1) is a desired property for any greedy f-flip solver that only seeks to improve the objective value with every move. Rearranging it, we can derive the following condition on M:

$$M > \frac{x'_{from}Qx_{from} - x'_{to}Qx_{to}}{(Ax_{to} - b)^2} \tag{2}$$

An exact calculation of the lower bound would involve minimizing the denominator of Eq. (2), since we are searching for n-dimensional binary vectors x_{to} such that $Ax_{to} = b+1$ or $b-1$ (in the best case) and we

do not have efficient methods to calculate the minimum denominator. Hence, we focus on maximizing the numerator with a selective sampling technique via a deterministic algorithm that guarantees a run time of $O(n^2)$. More specifically, we would choose solution vector transitions such that the value of the numerator of Eq. (2) is maximized.

Next, we will demonstrate how we choose such solution vectors for a greedy 1-flip solver. In order to maximize the numerator of Eq. (2), the following condition holds:

Remark 1. For a solver based on a local search using 1-flip neighborhood, the maximum change in the objective function at each iteration is given by:

$$maxChange = max\forall_{x_i}(\Delta(x'Qx)|_{x_i:0\to1}, \Delta(x'Qx)|_{x_i:1\to0})$$

where $\Delta()$ represents the change in the objective function as a result of each flip of variable x_i.

Note that for 1-flip solvers any variable x_i can be flipped from 0 to 1 and vice versa at every iteration. The first $\Delta()$ expression for a given x_i can be reduced to:

$$q_{ii} + \sum_{j\neq i}(q_{ij}x_j)$$

Similarly, the second $\Delta()$ expression is given by:

$$-q_{ii} - \sum_{j\neq i}(q_{ij}x_j)$$

For computing the estimate of lower bound of M, M_{est}, we would utilize solution vectors that maximize the value of the numerator of the inequality of M given by Eq. (2). Since we would prefer improving solution vectors as a result of transition i.e $x'_{to}Qx_{to} < x'_{from}Qx_{from}$, we are interested in maximizing both the $\Delta()$ expressions. These optimization problems can be easily evaluated by a one-pass of the Q matrix in $O(n)$ as follows:

$$max\ \Delta_1() = q_{ii} + max\ \sum_{j\neq i}(q_{ij}x_j) = q_{ii} + \sum_{j:(q_{ij}>0)\ \&\ (j\neq i)} q_{ij}$$

$$max\ \Delta_2() = -q_{ii} - \sum_{j:(q_{ij}<0)\ \&\ (j\neq i)} q_{ij}$$

Note that x_{from} and x_{to} could be easily derived while computing $\Delta_1()$ and $\Delta_2()$. More specifically, $x_j = 1$ in $\Delta_1()$ if $q_{ij} > 0$ and $x_j = 1$ in $\Delta_2()$ if $q_{ij} < 0$. The solution of these problems will yield the binary vectors that corresponds to the best improvement with the two possible bit flips ($0 \to 1$ & $1 \to 0$) for a specific variable x_i. Note that in order to obtain M_{est}, we need to solve these two optimization problems for each variable x_i. Our estimate for lower bound of M, M_{est}, is given by:

$$M_{est} = max\ \{q_{ii} + \sum_{(q_{ij}>0)}^{(j\neq i)} q_{ij}\ \forall i \in \{1,n\}, -q_{ii} - \sum_{(q_{ij}<0)}^{(j\neq i)} q_{ij}\ \forall i \in \{1,n\}\} \tag{3}$$

The total number of solution vectors is $2n$ for a problem size n. Hence the overall computational complexity to find M_{est} is $2{*}O(n^2)$. If the denominator of the expression for M in Eq. (2) is simultaneously minimized we could guarantee that the lower bound M_{est} is optimal. However, that would require a complete enumeration of the solution space. The hope is that the chosen solution vectors would result in a small change in the denominator $(Ax_{to} - b)^2$. Note that this calculation of denominator of Eq. (2) for 1-flip solvers is straightforward, given the solution vector x_{to}.

We will illustrate our technique to select the $2n$ solution vectors in Eq. (3) for M_{est} using the following example:

$$x'Qx = -x_1 + 2x_2 + 4x_3 - 4x_1x_2 + 6x_1x_3 - 2x_2x_3$$

For the variable x_1, the maximum value of Δ's is given by:

$$max\Delta_1() = -1 + 6 = 5$$
$$max\Delta_2() = 1 - (-4) = 5$$

The solution vectors for $\Delta_1()$ using transition of x_1 from 0 to 1 are $x_{from} = [0, 0, 1]$ and $x_{to} = [1, 0, 1]$. Note that for $\Delta_1() : x_1 : 0 \rightarrow 1$ and $x_j = 1\ \forall j \neq 1$ if $q_{ij} > 0$. If $A = \mathbf{1}$, $Ax_{from} = 1$ and $Ax_{to} = 2$. For $b = 1$, i.e. for an equality constraint $\sum_i x_i = 1$, the denominator is given by $(Ax_{to} - b)^2 = (2-1)^2 = 1$. Note that x_{from} is feasible while x_{to} is infeasible. Hence the first value of Eq. (3) i.e $\Delta_1()$ of x_1 is given by $\frac{5}{1} = 5$. For $\Delta_2()$ of x_1 we have $x_{from} = [1, 1, 0]$ and $x_{to} = [0, 1, 0]$. Note that for $\Delta_2() : x_1 : 1 \rightarrow 0$ and $x_j = 1\ \forall j \neq 1$ if $q_{ij} < 0$. Similarly, we can derive the other four initial estimates of x_2 and x_3. According to Eq. (3), our final estimate for the lower bound M_{est} is the maximum value of all the initial estimates for x_1, x_2 and x_3. These initial estimates would be discarded if the solution vector x_{to} is feasible i.e. $Ax_{to} = b$ because in such a case the denominator of Eq. (2) would be zero. Hence the initial estimate of M would be undefined.

3. Partitions of the slack range of a single inequality constraint

In this section, we will present a technique to achieve performance gains for QUBO having a single inequality constraint. The problem of interest is $min\ x'Qx\ \ s.t.\ \ Ax \leq b,\ \ x \in \{0,1\}$. The standard transformation converts the inequality to an equality constraint via additional binary slack variables to yield the unconstrained form $min\ xQx + M(Ax + Ds - b)^2$. Thus the original Q matrix is augmented to include the slack variables.

We could interpret the constraint $Ax \leq b$ as $Ax \in [0, b]$ assuming the coefficients of A and b are all positive integers. No positivity conditions are assumed for the Q matrix. In case a coefficient a_i of variable x_i is negative, we could work with the complement of x_i thereby transforming the Q matrix while maintaining the non-negative criteria of the A matrix. The number of required slack variables is controlled by the right hand side of the inequality constraint b such that as b increases, the number of slack variables increases. One simple way to cover the slack range of $[0, b]$ uses one slack variable for each integral point in the slack range. Thus the slack range is replaced by a linear combination of $b + 1$ slack variables as $Ax = 0 * s_0 + 1 * s_1 + 2 * s_2 + \cdots + b * s_b$. Only one of these slack variables could assume the value of 1 thereby requiring the constraint $\sum_{i=0}^{b} s_b = 1$. These two linear equalities could be easily transformed into the objective function as penalty terms $M * (Ax - \sum_{i=0}^{b} i\ s_b)^2$ and $M * (\sum_{i=0}^{b} s_b - 1)^2$ respectively. However, this increase in the number of slack variables need not be linear i.e. $O(b)$ with respect to b.

As a general rule, we want to limit the number of additional variables. In that regard, a better reformulation technique was explored in [11]. The authors were interested in choosing the minimal number of slack variables to cover the slack range $[0, b]$. They proposed a technique to cover a slack range corresponding to $2^0 + 2^1 + 2^2 + \cdots + 2^{n-1} = 2^n - 1$ units with only n slack variables. Their method relied on extending the existing slack range utilizing the coefficient values of different slack variables. For example, with one binary slack variable s_1, we could only cover the range of 1 unit. However, an additional slack variable s_2 involved in a term $2 * s_2$ allows us to extend the slack range to $1 + 2 = 3$ units. Thus, the number of slack variables was limited to $log_2(b+1)$ for the slack range $[0, b]$. For instance, if $b = 15$ we would use $Ds = s_1 + 2s_2 + 4s_3 + 8s_4$ thereby needing only 4 additional variables. If $b + 1$ is not an exact power of 2, we simply add a constant to both sides of the equation $Ax \leq b$ to make it so. At any rate, these additional variables increase the complexity of the QUBO model by increasing both Q density and total number of variables.

The landscape theory provides us tools to quantify the impact on performance of simple local search solvers due to additional variables. According to [12], the number of local optima N in a combinatorial landscape is given by the following expression:

$$N \approx \frac{|X|}{|X(x_o, l)|} \tag{4}$$

where $|X|$ denotes the total size of the solution space and $X(x_o, l)$ represents the set of solutions accessible from the vector x_o in l or less local moves. It is further shown that with a lower number of local optima, a local search solver could find a global optima in fewer steps. On the contrary, the problem becomes difficult to solve with a higher number of local optima. An estimation of N provided by [13] is given by:

$$N \approx \frac{2^n}{\sum_{i=0}^{l} \binom{n}{i}} \tag{5}$$

where n represents the size of the QUBO instance. Using this estimate, we could easily derive the following lemma:

Lemma 2. *As the size of the QUBO instance n increases, the estimate of the number of local optima increases.*

Proof. We need to prove that the expression of N provided in Eq. (5) is increasing in n. According to [14], the sum of first l binomial coefficient using a fixed n is given by:

$$\sum_{i=0}^{l} \binom{n}{i} \leq 2^{n-1} exp - \frac{(n-2l-2)^2}{4(1+l-n)}$$

Substituting this in Eq. (5), we have:

$$N \approx 2\, exp \frac{(n-2l-2)^2}{4(1+l-n)}$$

N is increasing in n as long as $log(N)$ is increasing in n. The first derivative of $log(N)$ is given by:

$$\frac{\partial log(N)}{\partial n} = \rho \frac{n(n-2l-2)}{(1+l-n)^2}$$

where ρ is a positive scalar. The denominator is a non-decreasing function in n. The numerator is given by a positive multiple of $n(n-2l-2)$. Hence N is increasing in n as long as $n \geq 2l+2$. In other words, for a sufficiently large n the lemma holds. □

Therefore as the size of the QUBO instance increases the number of local optima also increases thereby making the problem increasingly difficult to solve. We address this shortcoming for a single inequality constraint by employing a partitioning approach to divide the entire slack range into equal areas. Thus, we reduce the number of additional slack variables in each partition thereby reducing the size of each QUBO instance. The divide and conquer algorithmic paradigm has been successfully used in various areas of computer science [15]. The key difference between a classical divide and conquer algorithm and ours is that the subproblems in our case are independent from each other. We do not spend any significant computational time in collating the results from each partition. In our case, we simply report the lowest feasible value out of all the obtained solutions. However, we could design intelligent heuristics which share information among multiple partitions. We reserve such topics for future research.

More formally, the problem of interest is $min\ x'Qx\ s.t.\ Ax \leq b,\ x \in \{0,1\}$. As discussed earlier, we could easily recast the constraint as $Ax \in [0, b]$. Traditionally, $log_2(b+1)$ slack variables are required

to cover the slack range $[0, b]$. We propose m partitions such that partition i deals with the optimization problem $min\ x'Qx\ s.t.\ Ax \in [\frac{(i-1)(b+1)}{m}, \frac{i(b+1)}{m})$, $x \in \{0,1\}$. Each of these partitions require $O(log_2(\frac{(b+1)}{m}))$ slack variables. Thus, the partitioned problems require fewer slack variables. Moreover, these partitions are independent of each other and could be executed on multiple threads.

For instance, generally a right hand side of $b = 15$ needs 4 slack variables. Using our partitioning technique with $m = 4$, each of these partitions operate in a slack range of width 4 units thereby requiring only 2 slack variables. The lower and upper bounds for these 4 partitions are $[0, 3], [4, 7], [8, 11]$ and $[12, 15]$ respectively. For example, $Ax \in [4, 7]$ for the second partition. In other words, $Ax = 4+1*s_1+2*s_2$. The original objective function $min\ x'Qx$ is augmented with these five different penalty functions to yield five different optimization problems. In the case of the second partition, the penalty term is given by $M(Ax-4-s_1-2s_2)^2$. Thus the overall objective function is $min\ x'Qx + M(Ax-4-s_1-2s_2)^2$. In this way, each of these partitions would utilize only 2 slack variables compared to 4 slack variables for the original problem. This not only reduces the size of the QUBO problem but also the density of the Q matrix. It also provides an excellent candidate for multi-threading because each of these partitions can be solved independently. We will discuss the impact of this technique on the benchmark datasets in Section 4.2.

4. Computational experiments

For computational experiments, we use the ORLIB QUBO instances presented in [16]. These datasets have 2500 variables and 10% density. The integer coefficients are uniformly drawn from $[-100, 100]$. Similar to [6,7], we introduce a single cardinality constraint $\sum_i x_i = b$ where $b \in [100, 150, 200, 250, 500, 1000, 1500, 2000]$ as the linear equality to obtain an estimate of M. To test the performance of the partitioning routine, we introduce a simple linear inequality of the form $\sum_i x_i \leq b$. The algorithms for instance generation and solution technique were implemented in Python 3.6. The experiments were performed on a 3.40 GHz Intel Core i7 processor with 16 GB RAM running 64 bit Windows 7 OS.

We utilize a probabilistic multi-start greedy 1-flip solver based on path-relinking described in [17]. This solver has been benchmarked against CPLEX and a GRASP based path relinking solver [18] and proven to be highly effective. We start with a greedy initial solution by using an estimate of the contribution from each variable x_i given by $2q_{ii} + \sum_{i<j} q_{ij}$. If this estimate is greater than 0, $x_i = 1$. Thereafter, greedy one-flip moves, always selecting the variable providing the greatest contribution, are executed along with restarts based on probabilities of $x_i = \{0, 1\}$ and path relinking.

4.1. M results

For these experiments, we calculate the value of M_{est} according to Eq. (3) and present it in Table 1. The columns "Dataset" and "b" represent the dataset number and the right hand side of the constraint $\sum_{i=1}^{n} x_i = b$. As we can clearly observe from the results, the estimate for lower bound of M decreases as b increases. As b increases, approaching the cardinality of the optimal solution, the benefit of moving to an infeasible solution decreases, hence the magnitude of M needed to prevent infeasibility decreases. When b exceeds the cardinality of the optimal solution then the problem is unconstrained and $M = 0$ would suffice for optimal solutions. We note that non-optimal solutions may be infeasible, hence for practical reasons because optimality may not be found, a small penalty value (as opposed to zero) may be appropriate. Hence the constraint becomes trivial, especially for $\sum_i x_i = 2500$. We further note that typical choices of M do not change with the coefficient in the equality constraint [2]. On the contrary, we observed that the choice of M should depend on the value of b.

Next, we would want to validate this estimate of lower bound with computational experiments on the benchmark datasets. For this purpose, we ran the greedy 1-flip solver using $M \in [10, 100, 1000, M_{est}]$. The

Table 1
M_{est}.

Dataset	b								
	100	150	200	250	500	1000	1500	2000	2500
1	11 090	15 908	264	100	24	10	6	5	4
2	11 935	16 372	212	91	24	10	6	4	3
3	11 052	16 374	202	90	24	10	6	5	4
4	10 183	16 036	199	87	23	10	6	4	3
5	11 976	15 872	219	93	25	10	6	5	4
6	11 010	15 952	238	99	25	10	6	5	4
7	11 214	15 716	215	92	24	10	6	5	4
8	12 088	16 990	215	93	24	10	6	5	4
9	11 940	16 180	199	91	25	10	6	5	4
10	11 488	16 360	214	90	25	10	6	5	4

Table 2
$M = 100$ vs. $M = 1000$ vs $M = M_{est}$.

b	M = 100		M = 1000		M = M_{est}	
	100	150	100	150	100	150
1	[−74991 105]	[−122593 155]	[−69623, 101]	[−119192, 151]	[−66134 100]	[−113161 150]
2	[−77166 105]	[−125488 155]	[−70020, 100]	[−124326, 151]	[−67588 100]	[−113296 150]
3	[−74094 105]	[−121219 155]	[−71707, 101]	[−120093, 151]	[−66697 100]	[−111169 150]
4	[−72512 104]	[−124302 155]	[−71524, 101]	[−123314, 151]	[−66174 100]	[−110001 150]
5	[−75767 105]	[−125903 155]	[−70801, 100]	[−120753, 150]	[−67257 100]	[−114267 150]
6	[−75424 105]	[−121246 155]	[−65850, 100]	[−118201, 151]	[−65017 100]	[−109393 150]
7	[−74109 105]	[−129860 156]	[−68231, 100]	[−121548, 151]	[−65465 100]	[−111081 150]
8	[−73489 105]	[−123539 155]	[−67759, 100]	[−119354, 151]	[−66063 100]	[−111015 150]
9	[−74142 105]	[−119225 155]	[−68503, 101]	[−116798, 151]	[−65143 100]	[−110813 150]
10	[−76604 105]	[−124736 155]	[−71298, 101]	[−122317, 151]	[−67934 100]	[−113764 150]

complete set of results can be found in [19]. For higher values of b, the solver was not able to find any feasible solutions in the limited runtime for any value of M. These problems are increasingly difficult because the magnitude of the quadratic interactions overwhelms the search process. Moreover, a typical estimate of M is around 100. Since our estimates of M are close to 100 for $b \geq 200$, the more interesting case to explore is $b = 100$ and 150. We report the best solutions found by the solver in 500 s for $b = 100$ and 150 in Table 2 wherein we compare the performance of $M = 100, 1000$ and M_{est}. Note that we present the $[x'Qx, Ax]$ for the best found solution within 500 s for different values of b.

For all the datasets, a feasible solution is found with $M = M_{est}$ for the choices of b. As postulated, smaller values of b lead to a higher estimate of M. For instance, for $b = 100$ and 150 we observe that higher values of M are in fact needed. As evident from the results, if M is too small, like 100, it leads to infeasible solutions. As M increases to 1000, violating the constraint increases the penalty term. Hence, the solver does comparatively good in some $b = 100$ datasets. Again, the solver is unable to find feasible solutions with $b = 150$ in the majority of the cases. This emphasizes that higher values of M are desired in such settings and M_{est} could be really useful.

4.2. Partition results

Next we will investigate the results for partitioning of the slack range for a single inequality constraint $\sum_i x_i \leq b$. For the sake of brevity, we will present only a subset of the results. The complete set of results could be obtained at [19]. First, we compare the best solution for the full slack range with the best solution for the 5 partitions. For fair comparison, the QUBO model with full slack range was allowed to run for 2500 s whereas each of the 5 partitions were allotted only 500 s. We report the pair comprised of the objective function ($Obj\ Func\ : x'Qx$) and the left hand side of the constraint ($LHS\ : \sum_i x_i$) for the best found solution vector for $b = 250$ and $M \in [100, 10\,000]$ in Table 3.

Table 3
Partition results for $b = 250$.

Dataset	M = 100				M = 10 000			
	Full		Best partition		Full		Best partition	
	Obj func	LHS	Obj func	LHS	Obj func	LHS	Obj func	LHS
1	−239765	256	−196058	221	−386	3	−162422	200
2	−236185	256	−200890	221	−386	3	−165002	200
3	−234930	256	−193793	220	−464	3	−159089	200
4	−242181	256	−195522	221	−444	3	−158848	200
5	−240206	256	−193352	220	−441	3	−164080	200
6	−240569	256	−196444	221	−467	3	−157705	200
7	−238665	256	−196392	220	−366	3	−160369	200
8	−240922	256	−200890	221	−385	3	−161582	200
9	−241059	256	−195309	221	−388	3	−159759	200
10	−244302	256	−196205	221	−395	3	−163713	200

Table 4
Partition results for $Q4.txt$.

b	M = 100				M = 10 000			
	Full		Best partition		Full		Best partition	
	Obj func	LHS	Obj func	LHS	Obj func	LHS	Obj func	LHS
100	−74729	105	−68396	99	−373	3	−50151	80
150	−127263	155	−108786	140	−444	3	−82397	120
200	−181168	206	−152025	180	−444	3	−119837	160
250	−242181	256	−195522	221	−444	3	−158848	200
500	−546870	506	−440721	421	−303	3	−390329	400
1000	−1126303	1005	−947229	836	−806585	767	−570344	600
1500	−1470716	1501	−1320784	1235	−1376640	1919	−1480536	1583
2000	−1488347	1553	−1498097	1616	−1376640	1919	−1480536	1583
2500	−1463048	1509	−1479257	1516	−345849	2499	−1438497	1883

As we could clearly observe from the results in Table 3, the partitioning approach of the slack range consistently enhances the solution quality obtained by the solver. For $b = 250$ and $M = 100$, it was able to obtain feasible solutions for every instance. Moreover, it also improved the best feasible solution found by the solver for the full slack range with $M = 10\,000$. Next, we would investigate the solutions found for a specific dataset namely $Q4.txt$. This ORLIB dataset has 2500 nodes, 309 464 edges and 10% density. The integer coefficients are uniformly drawn from $[-100, 100]$. We provide the best solution found in the limited runtime in Table 4.

Once again, the partitioning approach improves the solution quality for all values of b for $M = 100$. It was always able to find a good feasible solution. For $M = 10\,000$, the partitioned approach still works better except for $b = 1000$ and 1500. In order to explain these set of results on partitions, we again resort to landscape theory. Our aim is to compare the difficulty of the full slack range instances with the partitioned slack range instances.

According to landscape theory, the difficulty of a QUBO instance is typically measured by the autocorrelation coefficient ξ proposed by [20]. This coefficient is tied to the number of local optima in the landscape (described in Section 3). The higher value of ξ corresponds to a lower number of local optima which could lead to better performance of the local search solver. More recently, the authors in [13] studied the combinatorial landscape of QUBO and derived an exact expression to compute ξ. However, their expression involves a computational complexity of $O(n^4)$. Therefore, it is applicable to only smaller instances. In the study, the authors presented results for QUBO instances with only 20 binary variables. Since we are interested in analyzing the performance of bigger datasets with 2500 variables, this approach is not suited. Therefore, we used the following expression of ξ proposed by [21]:

$$\xi = \frac{avg((f(x) - f(y))^2)_{(x,y)}}{avg((f(x) - f(y))^2)_{(x,y|d(x,y)=1)}} \tag{6}$$

Table 5
ξ for $Q4.txt$.

b	Full	Avg partition	% Diff
100	1126	1260	11.71
150	1335	1265	−5.27
200	881	1257	42.59
250	1037	1271	22.58
500	1676	1333	−20.49
1000	1005	1326	31.94
1500	842	1185	40.73
2000	1259	1191	−5.39
2500	790	1197	51.40

Here the numerator is the average value of $(f(x) - f(y))^2$ over all pairs of solution vectors x and y. The denominator restricts the solution vectors x and y to be at a distance of 1. In terms of Hamming distance, this means that x and y differ by exactly 1 bit. In other words, x and y are 1-flip neighbors for the denominator. The benefit of using this definition is that it can be easily estimated using Monte Carlo simulations (repeated random sampling) for our use case [22]. We used a sample size of 50 000 to get a better estimate.

We present the ξ results for $Q4$ in Table 5 to understand the performance gain of partitions for $Q4.txt$ and $M = 100$. The column $\%Diff$ represents the percentage difference between the average value of ξ among all the partitions (column $Avg\ Partition$) and the value of ξ corresponding to the full slack range (column $Full$). As we could observe from the results, the average ξ value for the partitioned case is significantly higher in 6 out of 9 instances. This explains the better performance achieved by partitioning of the slack range because higher ξ corresponds to fewer local optima thereby facilitating the local search solver. The ξ values for full slack range and partitioned slack range for the exceptions are within about 20% of each other. For one of the exceptions, namely, $b = 2000$, the individual ξ values for the 5 partitions are given by $[1360.36, 1103.82, 1328.04, 1116.11, 1046.03]$. Two out of these five values exceed the ξ for the case with full slack range which somewhat explains the better performance of local search in some partitions.

5. Conclusions

Driven by the NISQ-era need to reduce computational complexity of constraint transformed QUBO models, we presented and experimentally verified approaches for managing penalty magnitude for equality constraints in QUBO. We also presented an effective approach based on partitioning of the slack range that reduces the size of QUBO problems with a single inequality constraint. The impact on solution quality is shown to improve using benchmark QUBO problems with an additional cardinality constraint.

References

[1] G. Kochenberger, J.-K. Hao, F. Glover, M. Lewis, Z. Lü, H. Wang, Y. Wang, The unconstrained binary quadratic programming problem: a survey, J. Comb. Optim. 28 (1) (2014) 58–81.
[2] F. Glover, G. Kochenberger, Y. Du, Quantum bridge analytics i: a tutorial on formulating and using qubo models, 4OR 17 (4) (2019) 335–371.
[3] G.J. Mooney, S.U. Tonetto, C.D. Hill, L.C. Hollenberg, Mapping np-hard problems to restricted adiabatic quantum architectures, 2019, arXiv preprint arXiv:1911.00249.
[4] J. Preskill, Quantum computing in the nisq era and beyond, Quantum 2 (2018) 79.
[5] K. Cormier, R. Di Sipio, P. Wittek, Unfolding measurement distributions via quantum annealing, J. High Energy Phys. 2019 (11) (2019) 128.
[6] T. Vyskocil, H. Djidjev, Simple constraint embedding for quantum annealers, in: 2018 IEEE International Conference on Rebooting Computing, ICRC, IEEE, 2018, pp. 1–11.
[7] T. Vyskocil, H. Djidjev, Embedding equality constraints of optimization problems into a quantum annealer, Algorithms 12 (4) (2019) 77.
[8] T. Vyskočil, S. Pakin, H.N. Djidjev, Embedding inequality constraints for quantum annealing optimization, in: International Workshop on Quantum Technology and Optimization Problems, Springer, 2019, pp. 11–22.

[9] Z. Bian, F. Chudak, R.B. Israel, B. Lackey, W.G. Macready, A. Roy, Mapping constrained optimization problems to quantum annealing with application to fault diagnosis, Front. ICT 3 (2016) 14.
[10] A. Verma, M. Lewis, Optimal quadratic reformulations of fourth degree pseudo-boolean functions, Optim. Lett. (2019) 1–13.
[11] F. Glover, G. Kochenberger, B. Alidaee, M. Amini, Solving quadratic knapsack problems by reformulation and tabu search: Single constraint case, in: Combinatorial and Global Optimization, World Scientific, 2002, pp. 111–121.
[12] P.F. Stadler, Fitness landscapes, in: Biological Evolution and Statistical Physics, Springer, 2002, pp. 183–204.
[13] F. Chicano, E. Alba, Elementary landscape decomposition of the 0-1 unconstrained quadratic optimization, J. Heuristics 19 (4) (2013) 711–728.
[14] L. Lovász, J. Pelikán, K. Vesztergombi, Diskrete Mathematik, Springer-Verlag, 2006.
[15] G. Bontempi, M. Birattari, From linearization to lazy learning: a survey of divide-and-conquer techniques for nonlinear control, Int. J. Comput. Cogn. 3 (1) (2005).
[16] J.E. Beasley, Or-library: distributing test problems by electronic mail, J. Oper. Res. Soc. 41 (11) (1990) 1069–1072.
[17] M. Lewis, G. Kochenberger, Probabilistic multistart with path relinking for solving the unconstrained binary quadratic problem, Int. J. Oper. Res. 26 (1) (2016) 13–33.
[18] Y. Wang, Z. Lü, F. Glover, J.-K. Hao, Path relinking for unconstrained binary quadratic programming, European J. Oper. Res. 223 (3) (2012) 595–604.
[19] Results, 2019, https://github.com/amitverma1509/DiscreteOptimization2019. (Accessed 12 December 2019).
[20] E. Angel, V. Zissimopoulos, Autocorrelation coefficient for the graph bipartitioning problem, Theoret. Comput. Sci. 191 (1–2) (1998) 229–243.
[21] E. Weinberger, Correlated and uncorrelated fitness landscapes and how to tell the difference, Biol. Cybern. 63 (5) (1990) 325–336.
[22] N. Metropolis, S. Ulam, The monte carlo method, J. Amer. Statist. Assoc. 44 (247) (1949) 335–341.